



TDD

Introduktion till Test Driven Utveckling

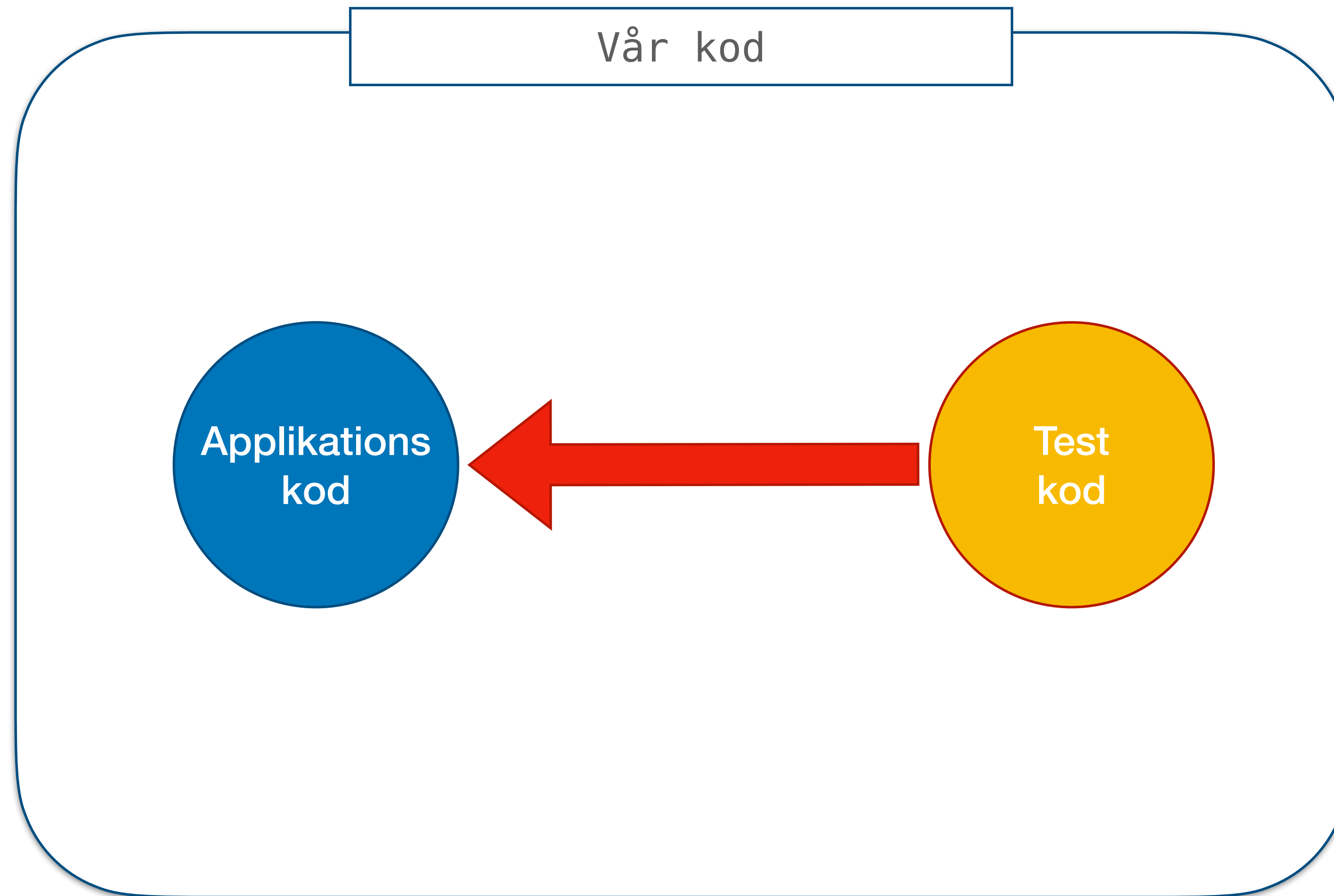


- I denna sektion ska vi gå igenom Test Driven Utveckling
 - Vad är Test Driven Utveckling?
 - Varför?
 - Olika typer av tester
 - Hur gör vi?

Vad är test drivet?

- Vanan att skriva kod som testar vår kod
- Samt att vi kör tester automatiserat
 - Vi ska inte behöva skriva console.log kommandon
 - Vilket vi ofta gör 😊
 - Istället använder vi test motorer

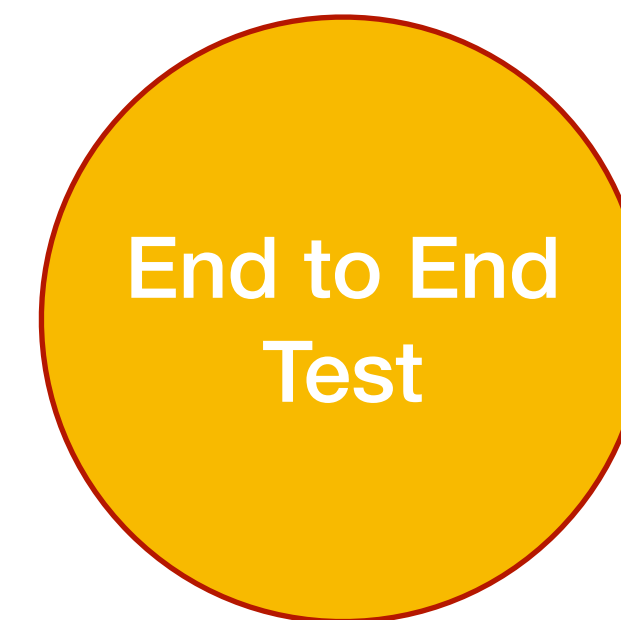
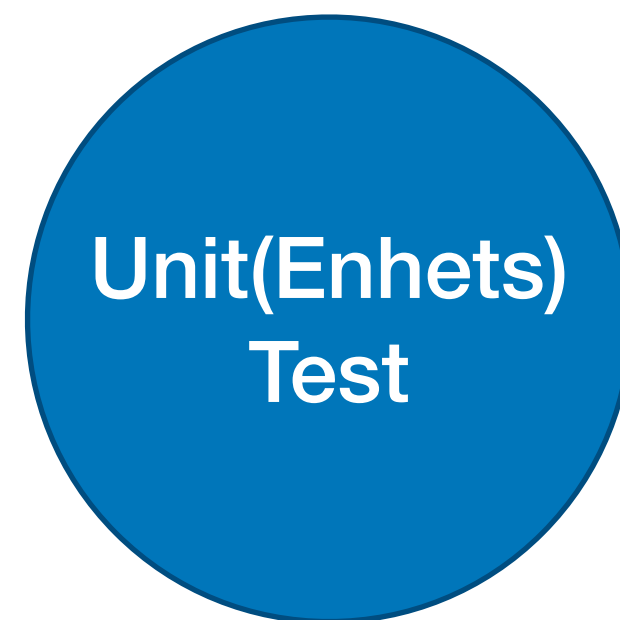
Vad är test drivet forts...



Varför?

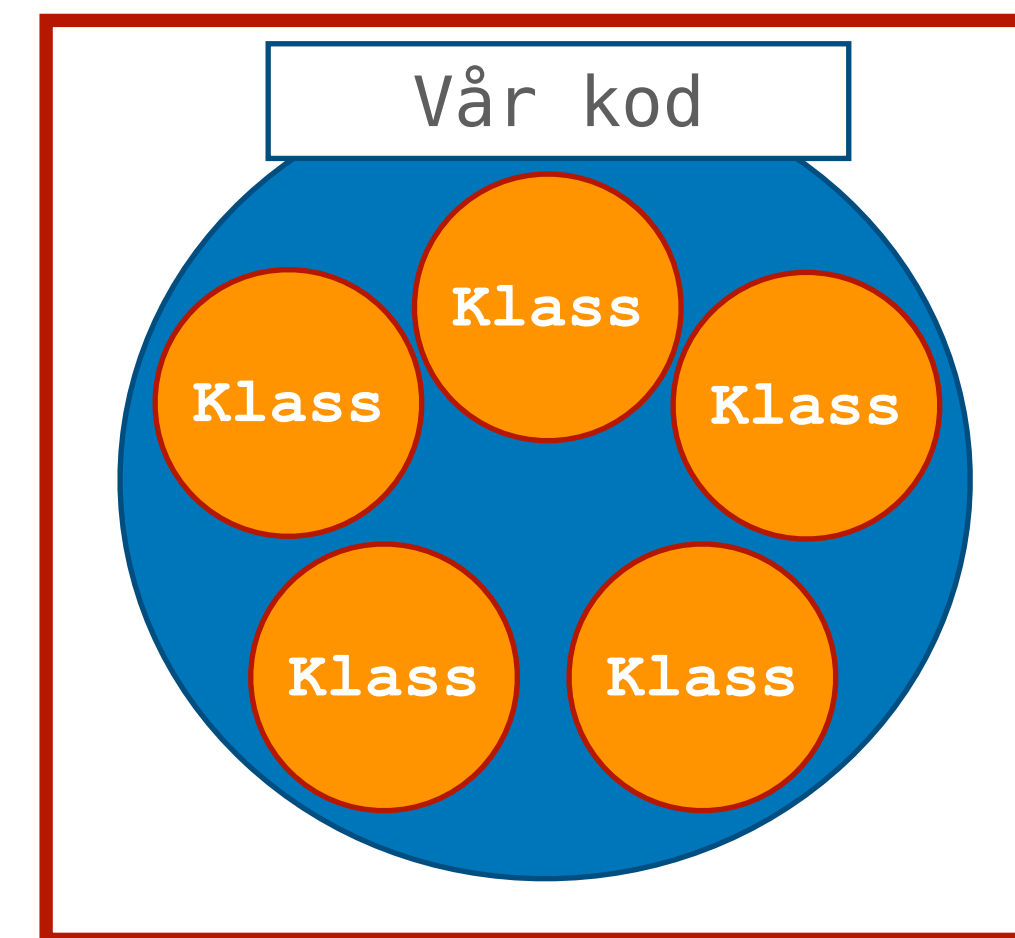
- Lansera en buggfri applikation
 - Tester låter oss fånga eventuella buggar innan produktions lansering
- Automatiserad testning är snabbt
 - Gör att vi kan testa kod ofta och snabbt
- Manuell testning är långsamt och tidskrävande
 - Kostar mer att göra
- Förändringar i kod(refactoring) blir lättare att göra
 - Testerna garanterar att problem eller buggar upptäcks snabbt och innan lansering
- Leverera med självförtroende och kvalitet

Olíka typer av tester



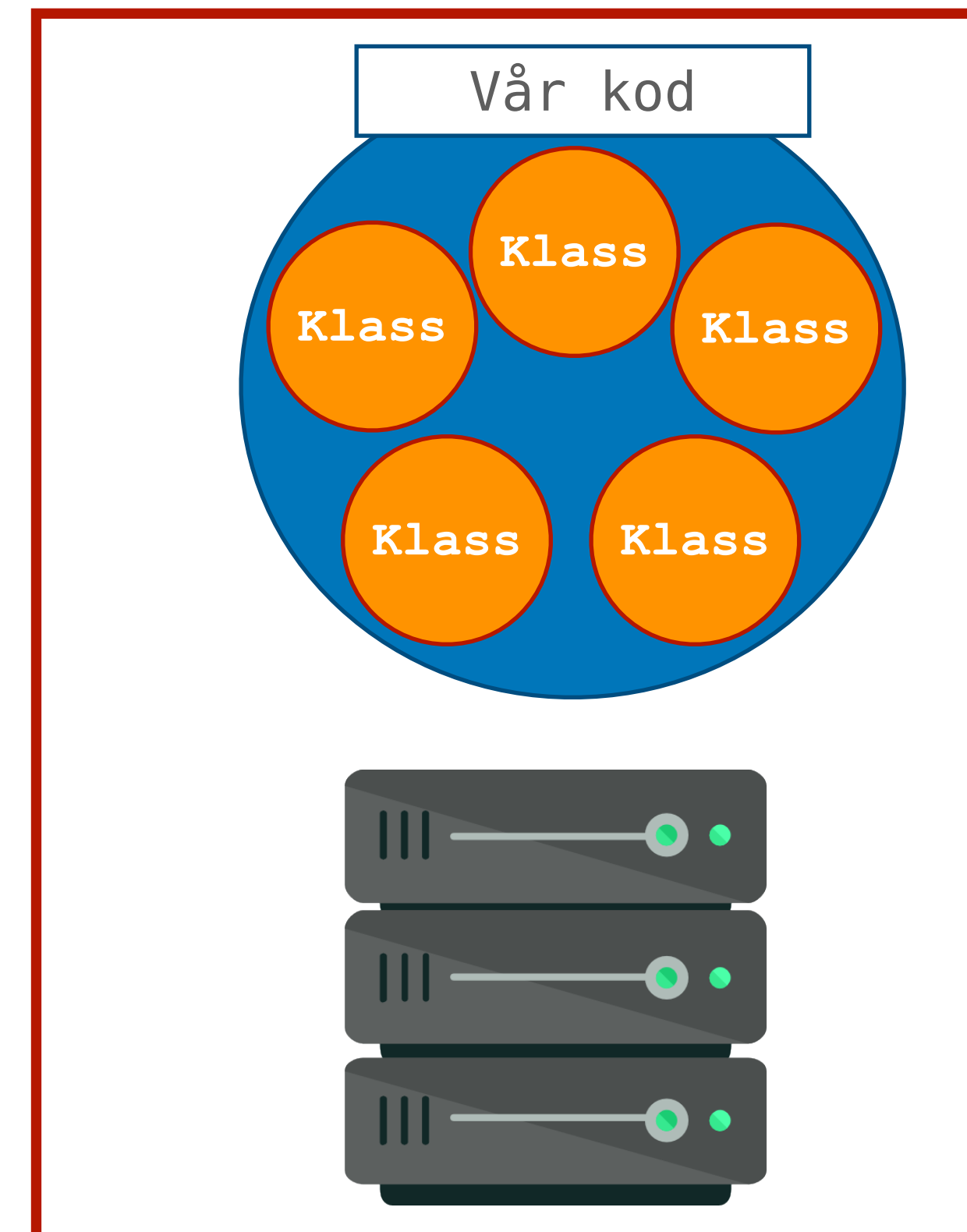
Unit Test

- En test av en avgränsad del av en applikation utan externt beroende
 - Isolerad testning
- Enkel att skriva
- Snabb



Integrerings Test

- En test av en applikation med dess externa beroende
- Något långsammare
 - Tar längre tid att köras



End to End Test

- Testar alla delar i en applikation

- Ifrån UI
- Via komponenter(klasser)
- Till backend(databaser, filer osv...)
- Och tillbaka

- Selenium

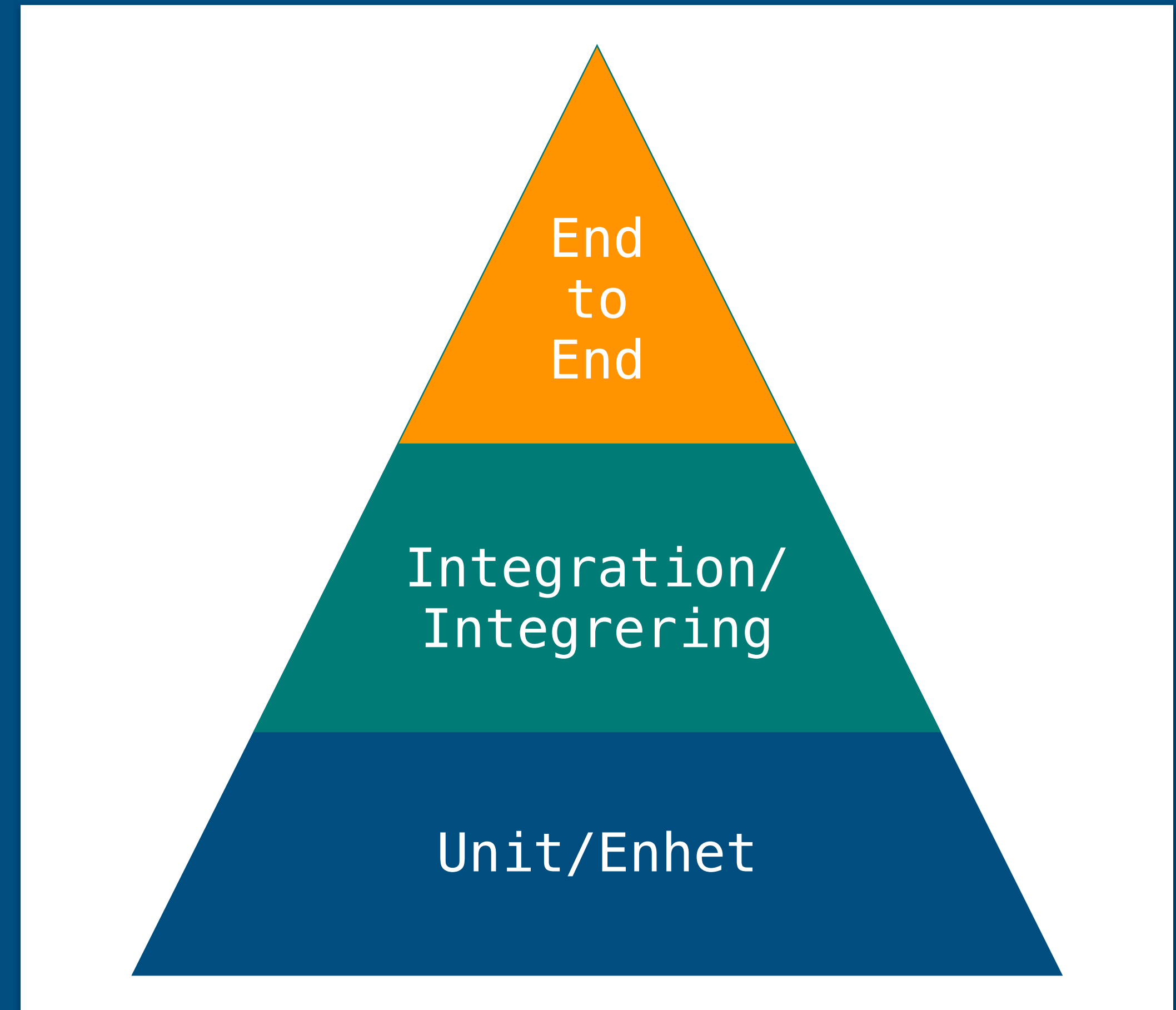
- Är ett populärt verktyg för End to End testning
- Låter oss spela in en användares användande av en applikation

OBSERVERA!
End-to-End tester är väldigt långsamma att köra
så oftast har vi färre av dessa och endast kör dem vid behov

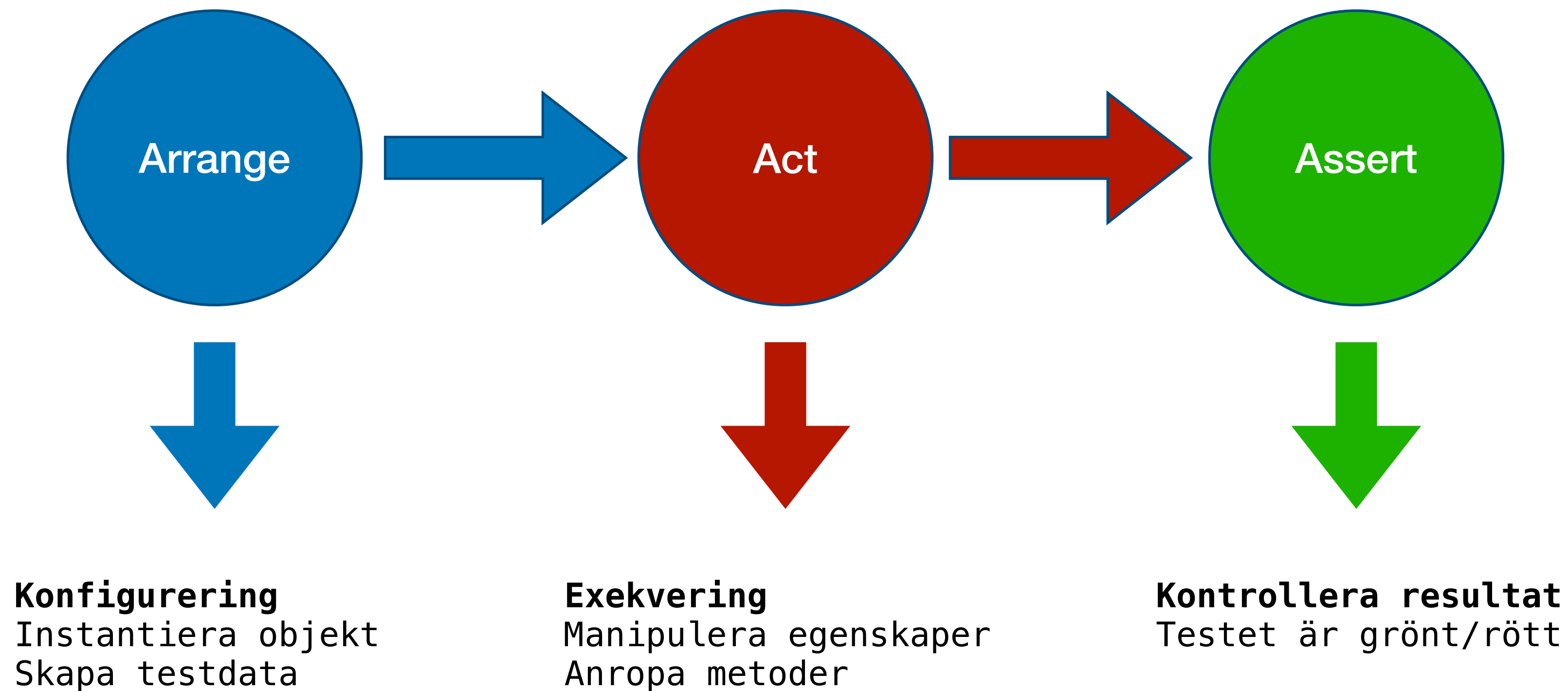


The Pyramid of Test

- Fokusera på enhets tester
 - Snabba att skriva och köra
 - Oftast många
- Integreringstester
 - För att testa integrering med externa beroenden
 - Viktiga, men oftast inte lika många som enhetstester
- End to End
 - Viktiga men är långsamma att köra
 - Oftast endast ett fåtal per applikation



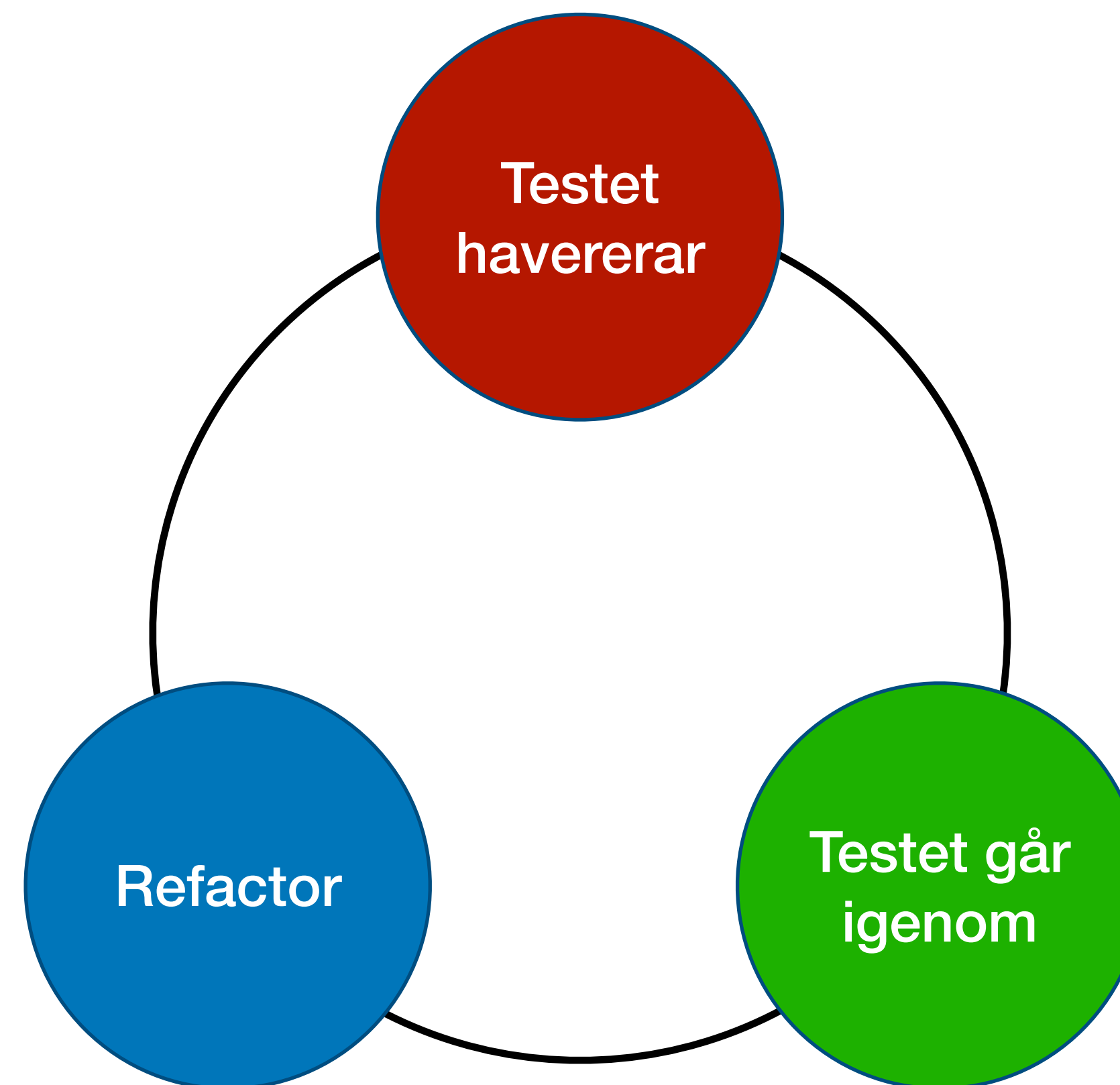
Logiska faserna i en test



Arbetsätt

Red & Green

1. Skriv test, ska inte fungera
2. Skriv kod som går igenom
3. Refactor kod
4. Repetera steg 1 och 2



Test ramverk

- Populära test ramverk för JavaScript, TypeScript och React
 - Jest
 - Vitest, vilket vi kommer att använda
 - Jasmine
 - Mocha

Demo



- Låt oss nu se hur vi sätta upp vår miljö för testdriven utveckling.